

Projektowanie Leków -- Uczenie Maszynowe

Przygotowanie i importowanie pakietów

Wykonaj poniższe komórki, żeby zainstalować bibliotekę RDKit i zaimportować wszystkie potrzebne pakiety

```
!pip install rdkit

Requirement already satisfied: rdkit in
/usr/local/lib/python3.12/dist-packages (2025.9.4)
Requirement already satisfied: numpy in
/usr/local/lib/python3.12/dist-packages (from rdkit) (2.0.2)
Requirement already satisfied: Pillow in
/usr/local/lib/python3.12/dist-packages (from rdkit) (11.3.0)

import os

from typing import Dict

import rdkit
from rdkit import Chem
from rdkit import RDConfig
from rdkit.Chem.FeatMaps import FeatMaps
from rdkit.Chem import Draw, AllChem, rdMolAlign

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Pobranie Danych

Dane pobierzemy z internetu z użyciem komendy `wget`

```
!wget https://raw.githubusercontent.com/dgront/chem-ml/refs/heads/main/INPUTS/tdcommons-data/cypla2_veith.tab

--2026-02-09 23:12:20--
https://raw.githubusercontent.com/dgront/chem-ml/refs/heads/main/INPUTS/tdcommons-data/cypla2_veith.tab
Resolving raw.githubusercontent.com (raw.githubusercontent.com)...
185.199.109.133, 185.199.110.133, 185.199.108.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|
185.199.109.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 760372 (743K) [text/plain]
Saving to: 'cypla2_veith.tab.1'
```

```
cypla2_veith.tab.1 100%[=====>] 742.55K --.-KB/s in
0.05s
```

```
2026-02-09 23:12:20 (15.3 MB/s) - 'cypla2_veith.tab.1' saved
[760372/760372]
```

Wykonaj poniższą komendę, żeby wyświetlić początek pliku

```
!head cypla2_veith.tab
```

```
Drug_ID Drug_Y
6602638.0 "CCCC(=O)Nc1ccc(N2CCN(CC)CC2)c(Cl)c1.Cl" 0
644510.0 "O=c1[nH]c2cc3c(cc2cc1CN(CCCO)Cc1nnnn1Cc1ccc(F)cc1)OC03"
1
1960010.0 "CCN1C(=O)/C(=C2\SC(=S)N(CCCOC)C2=O)c2ccccc21" 1
644675.0 "CC(=O)N(c1ccc2oc(=O)sc2c1)S(=O)(=O)c1cccs1" 1
644851.0 "Clc1cccc1-c1nc(-c2ccccc2)n[nH]1" 1
644968.0 "C0c1cccc1CNC(=O)Cn1nnc(-c2ccncc2)n1" 1
6602688.0 "Br.N=c1n(CCN2CCOCC2)c2ccccc2n1CC(=O)c1ccc(Cl)c(Cl)c1" 1
5768708.0 "C0c1ccc(/C(0)=C2/C(=O)C(=O)N(CCCC(=O)O)C2c2ccc(OC)cc2)cc1"
0
645323.0 "C0c1ccc(Nc2nc(N)nc(CSc3n[nH]c(-c4ccccc4)n3)n2)cc1" 1
```

Zadanie 1: Załadowanie i przygotowanie danych

Zadanie 1.0

Wczytaj dane z pobranego pliku `./data.tab` do obiektu `data` typu `DataFrame` (z biblioteki `pandas`). Użyj w tym celu funkcji `read_csv()`. Nazwij kolumny `id`, `smiles` i `label`. Wyświetl tabelę

```
data = pd.read_csv('cypla2_veith.tab', sep='\t', names=['id',
'smiles', 'label'], skiprows=1)
print(data.shape)

(12579, 3)
```

Zadanie 1.1

Wyświetl 10 pierwszych wierszy z `data`

```
display(data.head(n=11))

{"summary":{"\n  \"name\": \"display(data\", \n  \"rows\": 11, \n
\"fields\": [\n    {\n      \"column\": \"id\", \n      \"properties\":
{\n        \"dtype\": \"number\", \n        \"std\":
```

```

2614247.648760364,\n          \"min\": 644510.0,\n          \"max\":  

6602688.0,\n          \"num_unique_values\": 11,\n          \"samples\":  

[\n          644968.0,\n          6602638.0,\n          645573.0\n],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n}\n      },\n      {\n          \"column\": \"smiles\",\n          \"properties\":  

{\n          \"dtype\": \"string\",\n          \"num_unique_values\": 11,\n          \"samples\": [\n          \"C0c1ccccc1CNC(=O)Cn1nnc(-  

c2ccncc2)n1\",\n          \"CCCC(=O)Nc1ccc(N2CCN(CC)CC2)c(Cl)c1.Cl\",\n          \"CCN(CC)C(=O)CSc1nnc(-c2cc3ccccc3cc20)n1CC\"\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n      }  

},\n      {\n          \"column\": \"label\",\n          \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 0,\n          \"min\":  

0,\n          \"max\": 1,\n          \"num_unique_values\": 2,\n          \"samples\": [\n          1,\n          0\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n      }  

}\n    ]\n  },\n  \"type\": \"dataframe\"}

```

Zadanie 1.2

Łaďaduj do RDKit molekuly z wierszy 0 i 10000 i wyświelt ich wzory strukturalne

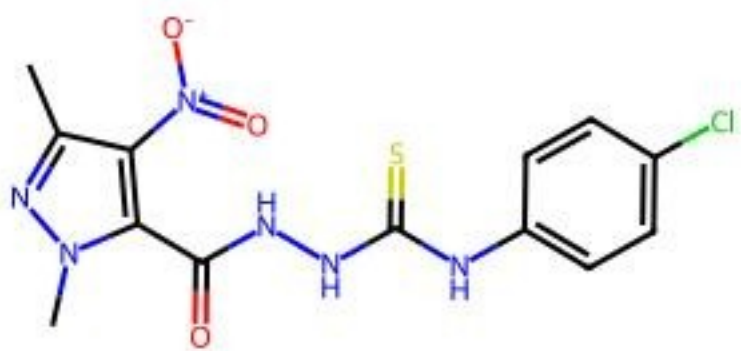
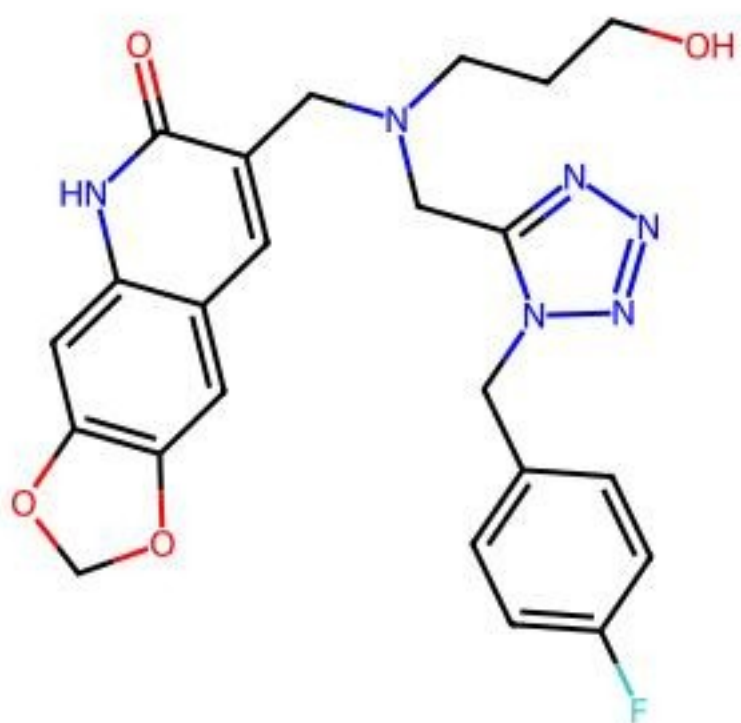
```

mol0_smiles = data.loc[1, 'smiles']
mol10000_smiles = data.loc[10000, 'smiles']

molecules_0 = rdkit.Chem.MolFromSmiles(mol0_smiles)
molecules_10000 = rdkit.Chem.MolFromSmiles(mol10000_smiles)

display(Draw.MolToImage(molecules_0))
display(Draw.MolToImage(molecules_10000))

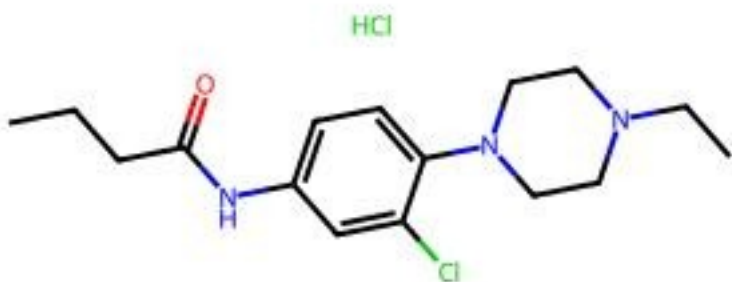
```



Zadanie 1.3

Utwórz listę `molecules` zawierającą molekuly utworzone ze SMILES-ów z data z wykorzystaniem RDKit

```
molecules = [rdkit.Chem.MolFromSmiles(smiles) for smiles in data['smiles']]  
molecules = [mol for mol in molecules if mol is not None]  
  
# Sanity check  
display(Draw.MolToImage(molecules[0]))
```



Zadanie 2: Uzyskanie *features* dla jednej molekuly

Sieć neuronowa potrzebuje odpowiednio przygotowanych danych. My zbudujemy bardzo prostą sieć, która na wejściu przyjmować będzie:

- liczby poszczególnych atomów
- liczby określonych *features* z RDKit'a

Przyjmijmy, że wektor dla pojedynczej molekuly będzie mieć na kolejnych miejscach liczby odpowiednio:

- donorów
- akceptorów

- centrów hydrofobowych
- atomów węgla
- atomów azotu
- atomów tlenu
- atomów wodoru

Zadanie 2.1

Zapisz pierwszą (o indeksie 0) molekułę z `molecules` do zmiennej `mol0`

```
mol0 = molecules[0]
```

Zadanie 2.2

Korzystając z funkcji `AllChem.BuildFeatureFactory()` zbuduj obiekt `feature_factory` i wypisz listę wszystkich chemicznych *features* (patrz: ostatnie zajęcia)

```
feature_factory =
AllChem.BuildFeatureFactory(os.path.join(RDConfig.RDDataDir, 'BaseFeatu
res.fdef'))
features = feature_factory.GetFeaturesForMol(mol0)

for ftype in feature_factory.GetFeatureFamilies():
    print(ftype)

Donor
Acceptor
NegIonizable
PosIonizable
ZnBinder
Aromatic
Hydrophobe
LumpedHydrophobe
```

Zadanie 2.3

Wyznacz i wypisz liczbę poszczególnych chemicznych features w `mol0`

```
features_mol0 = feature_factory.GetFeaturesForMol(mol0)
feature_counts: Dict = {}

for feat in features_mol0:
    feature_type = feat.GetFamily()
    feature_counts[feature_type] = feature_counts.get(feature_type, 0)
    + 1

for ftype, count in feature_counts.items():
    print(f"{ftype}: {count}")
```

Donor: 2
Acceptor: 1
PosIonizable: 1
Aromatic: 1
Hydrophobe: 3
LumpedHydrophobe: 1

Zadanie 2.4

Wyznacz i wypisz liczbę atomów węgla, tlenu, azotu, fluoru, siarki i chloru w cząteczce `mol`

```
atom_counts = {"C": 0, "O": 0, "N": 0, "F": 0, "S": 0, "Cl": 0}

for atom in mol0.GetAtoms():
    symbol = atom.GetSymbol()
    if symbol in atom_counts:
        atom_counts[symbol] += 1

for symbol, count in atom_counts.items():
    print(f"{symbol}: {count}")

C: 16
O: 1
N: 3
F: 0
S: 0
Cl: 2
```

Zadanie 3: Przygotowanie danych wejściowych

Bazując na poprzednim zadaniu przygotuj dane wejściowe do sieci. Dla każdej molekuły powinieneś mieć odpowiednie liczby *features* z RDKit'a oraz liczby określonych atomów. Jednocześnie powinieneś mieć ułożone w tej samej kolejności etykiety (ostatni rząd w `data`).

Dane możesz umieścić np w dwuwymiarowej liście, `ndarray`, albo po prostu dodać kolumny z nowymi parametrami do `data`. Póki co nie rozdzielaj danych wejściowych i etykiet, bo będzie je jeszcze trzeba przemieszać i podzielić na dane treningowe i testowe.

```
def is_ring_aromatic(mol, bond_ring_indices):
    """ Checks if a specific ring (defined by bond indices) is
    aromatic.
    Returns True if ALL bonds in the ring are aromatic.
    """
    for bond_idx in bond_ring_indices:
        if not mol.GetBondWithIdx(bond_idx).GetIsAromatic():
            return False
    return True
```

```

def get_ring_features(mol):
    ri = mol.GetRingInfo()
    bond_rings = ri.BondRings()
    num_rings = ri.NumRings()
    num_aromatic = 0
    for ring in bond_rings:
        if is_ring_aromatic(mol, ring):
            num_aromatic += 1

    num_aliphatic = num_rings - num_aromatic

    return {
        "Num_Rings_Total": num_rings,
        "Num_Rings_Aromatic": num_aromatic,
        "Num_Rings_Aliphatic": num_aliphatic
    }

x_map = {
    'atomic_num': list(range(0, 119)),
    'chirality': [
        'CHI_UNSPECIFIED', 'CHI_TETRAHEDRAL_CW',
        'CHI_TETRAHEDRAL_CCW', 'CHI_OTHER',
        'CHI_TETRAHEDRAL', 'CHI_ALLENE', 'CHI_SQUAREPLANAR',
        'CHI_TRIGONALBIPYRAMIDAL', 'CHI_OCTAHEDRAL',
    ],
    'degree': list(range(0, 11)),
    'formal_charge': list(range(-5, 7)),
    'num_hs': list(range(0, 9)),
    'num_radical_electrons': list(range(0, 5)),
    'hybridization': [
        'UNSPECIFIED', 'S', 'SP', 'SP2', 'SP3', 'SP3D', 'SP3D2',
        'OTHER',
    ],
    'is_aromatic': [False, True],
    'is_in_ring': [False, True],
}

e_map = {
    'bond_type': [
        'UNSPECIFIED', 'SINGLE', 'DOUBLE', 'TRIPLE', 'QUADRUPLE',
        'QUINTUPLE', 'HEXTUPLE',
        'ONEANDAHALF', 'TWOANDAHALF', 'THREEANDAHALF', 'FOURANDAHALF',
        'FIVEANDAHALF',
        'AROMATIC', 'IONIC', 'HYDROGEN', 'THREECENTER', 'DATIVEONE',
        'DATIVE',
        'DATIVEL', 'DATIVER', 'OTHER', 'ZERO',
    ],
    'stereo': [
        'STEREONONE', 'STEREOANY', 'STEREOZ', 'STEREOE', 'STEREOCIS',

```



```

'STEREOTRANS',
    ],
    'is_conjugated': [False, True],
}

chem_features_names = ["Donor", "Acceptor", "NegIonizable",
                       "Hydrophobe", "PosIonizable", "ZnBinder",
                       "Aromatic", "LumpedHydrophobe"]
chem_cols = [f"{feat}_Count" for feat in chem_features_names]

# Ring Features
ring_cols = ["Num_Rings_Total", "Num_Rings_Aromatic",
             "Num_Rings_Aliphatic"]
structural_cols = []
for key, values in x_map.items():
    for val in values:
        structural_cols.append(f"Atom_{key}_{val}")
for key, values in e_map.items():
    for val in values:
        structural_cols.append(f"Bond_{key}_{val}")

all_columns = chem_cols + ring_cols + ["Formal_Charge_Sum"] +
structural_cols
processed_features_list = []

for mol in molecules:
    row_dict = {col: 0 for col in all_columns}

    if mol:
        feats = feature_factory.GetFeaturesForMol(mol)
        for feat in feats:
            ftype = feat.GetFamily()
            col_name = f"{ftype}_Count"
            if col_name in row_dict:
                row_dict[col_name] += 1

        ring_data = get_ring_features(mol)
        for k, v in ring_data.items():
            row_dict[k] = v

        row_dict["Formal_Charge_Sum"] = Chem.GetFormalCharge(mol)

        for atom in mol.GetAtoms():
            row_dict[f"Atom_atomic_num_{atom.GetAtomicNum()}"] += 1
            # Chirality
            row_dict[f"Atom_chirality_{str(atom.GetChiralTag())}"] +=
1
            # Degree
            row_dict[f"Atom_degree_{atom.GetDegree()}"] += 1

```

```

        # Formal Charge (Map specific counts)
        f_chg = atom.GetFormalCharge()
        if f_chg in x_map['formal_charge']:
            row_dict[f"Atom_formal_charge_{f_chg}"] += 1

        # Num Hs
        row_dict[f"Atom_num_hs_{atom.GetTotalNumHs()}"] += 1
        # Radical Electrons

row_dict[f"Atom_num_radical_electrons_{atom.GetNumRadicalElectrons()}"]
] += 1

        # Hybridization

row_dict[f"Atom_hybridization_{str(atom.GetHybridization())}"] += 1
        # Is Aromatic
        row_dict[f"Atom_is_aromatic_{atom.GetIsAromatic()}"] += 1
        # Is In Ring
        row_dict[f"Atom_is_in_ring_{atom.IsInRing()}"] += 1

        # Bond Properties
        for bond in mol.GetBonds():
            row_dict[f"Bond_bond_type_{str(bond.GetBondType())}"] += 1
            row_dict[f"Bond_stereo_{str(bond.GetStereo())}"] += 1
            row_dict[f"Bond_is_conjugated_{bond.GetIsConjugated()}"]
+= 1

        row_data = [row_dict[col] for col in all_columns]
        processed_features_list.append(row_data)

feature_df = pd.DataFrame(processed_features_list,
                           columns=all_columns)
labels_for_molecules =
data["label"].astype(int).reset_index(drop=True)
feature_df["label"] = labels_for_molecules
input_data_df = feature_df

display(input_data_df.head(n=5))
print(f"Generated DataFrame shape: {input_data_df.shape}")

{"type": "dataframe"}
Generated DataFrame shape: (12579, 220)

```

Zadanie 3.1

Obejrzyj uzyskaną tabelę danych; sprawdź czy wszystko wygląda tak jak powinno

```

display(input_data_df.head())

{"type": "dataframe"}

```

Zadanie 3.2

Zapisz dane do pliku `data.csv`

```
input_data_df.to_csv('data.csv', index=False)
```